CERTAMEN 1
CICD
Instrucciones:

- Lea con calma las instrucciones antes de responder.
- La prueba es personal. Copiar, dejarse copiar o pasar información a otra persona se considera falta grave.
- Puede buscar en Internet y apoyarse en los Dockerfiles y docker-compose.yml vistos en clases.
- **Entregue su trabajo por el aula, en la tarea destinada para ello.**
- **No se aceptan entregas atrasadas.**
- **Las respuestas serán revisadas con una plataforma de detección de uso de IA. Si el resultado supera el 40%, tendrá que responder una interrogación para demostrar que domina la solución que entregó.**

1. Generar el Dockerfile y el comando docker run para desplegar el siguiente código fuente (carpeta `Certamen12026/ej1`, API en Node.js/Express que usa MongoDB):

Requisitos:

- El Dockerfile debe dejar el código fuente inmutable dentro de la imagen.
- Entregar el comando docker run completo, con las variables de entorno que la app necesita.
- El contenedor debe quedar expuesto en el puerto 8089.
- La imagen se debe llamar “cert1_2026” con tag “1.0”.
- Incluir el comando para levantar el contenedor de MongoDB en caso de que no exista.

2. Definir el Dockerfile y el comando docker run para desplegar el siguiente código fuente (carpeta `Certamen12026/ej2`, API en NestJS con TypeScript):

Requisitos:

- El Dockerfile debe permitir ejecutar la aplicación a partir del fuente que está en el host.

- El contenedor debe quedar expuesto en el puerto 9595.

- El código fuente se debe vincular al contenedor con un **bind mount**, de modo que los cambios hechos en el host se vean reflejados en el contenedor.

- Entregar el comando docker run completo, usando una imagen base adecuada para correr NestJS.

- Pasar correctamente las variables de entorno necesarias en el comando.

3. Definir la orquestación con Docker Compose para el siguiente código fuente (carpeta `Certamen12026/ej3`, API en Spring Boot con Java 17 y Maven que guarda sus datos en MySQL):

Requisitos:

- El código fuente debe ser inmutable (se sugiere un Dockerfile multi-stage: compilar el JAR en una etapa y ejecutarlo en una imagen liviana).

- Desplegar dos servicios de la aplicación, uno en el puerto 9595 y otro en el 9598.

- Definir la política de reinicio: unless-stopped.

- Generar el contenedor de base de datos MySQL (usar volumen para persistir los datos).

- Los servicios de la aplicación no deben iniciar hasta que MySQL esté listo.

- Los servicios deben implementar una red isolada por cada entorno definida en el docker-compose.yml.

Consideraciones Generales:

- Entregar el certamen a través de la tarea del aula, en un solo archivo ZIP (por ejemplo `certamen1.zip`).
    - Dentro del ZIP debe haber una carpeta `certamen1` y, dentro de ella, una carpeta por ejercicio (`ej1`, `ej2`, `ej3`), con esta estructura:

```
certamen1/
├── ej1/
│   ├── Dockerfile
│   └── comandos.txt
├── ej2/
│   ├── Dockerfile
│   └── comandos.txt
└── ej3/
    ├── Dockerfile
    ├── docker-compose.yml
    └── comandos.txt
```

- Cada ejercicio lleva un archivo txt (`comandos.txt`) con los comandos docker que se piden (build, run, compose, etc.), además de sus Dockerfiles.

- El docker-compose.yml va solo en la carpeta `ej3`.

CORRECCIÓN

Ítem	Puntos	Obtenidos
Dockerfile 1	10	
Comandos requeridos 1	15	
Dockerfile 2	10	
Comandos requeridos 2	15	
Orquestacion 3	30	
Dockerfile 3	10	
Comunicacion entre contenedores ejercicio 3	10	